HANDS-ON TUTORIAL

AI-Assisted Legacy Application Modernization with Kiro

Reverse-engineer three legacy .NET applications, rebuild their business architecture from evidence, grade the result against an answer key, and forward-engineer them into one modern web application, all driven by Kiro.

> **Required tool: Kiro** (https://kiro.dev). Every stage in this tutorial is executed by the Kiro AI agent. Install Kiro and sign in before you begin.

Legacy stack: .NET 10 WPF desktop, ASP.NET Core MVC, ASP.NET Core Web API, SQL Server
Target: ASP.NET Core .NET 10 modular monolith (UnifiedRetail)
Companion repository: retail-portfolio-modernization

Contents

About this tutorial 4
What you will learn 4
What you will end up with 4
Time required 4
Kiro: the operative tool 5
What Kiro is 5
Why this exercise depends on Kiro 5
Kiro concepts used in this tutorial 5
The scenario: a legacy retail portfolio 7
Why this portfolio is a realistic test 7
The target 8
The six-stage method 9
Principles that make the method work 9
Module 1: Set up your environment 11
1.1 Prerequisites 11
1.2 Install and sign in to Kiro 11
1.3 Get the repository and build it 11
1.4 Provision the database 11
1.5 Run the legacy applications 12
1.6 Run the legacy tests 12
1.7 Open the repository in Kiro 12
Module 2: Prepare a clean learner workspace 13
2.1 Create your learner branch and the reference copy 13
2.2 Strip the learner branch down to the legacy code 13
2.3 Optional: give Kiro trusted commands 13
Module 3: How to run a stage in Kiro (repeat for every stage) 15
3.1 Comparing your results with the reference 15
3.2 Quick validity checks you can run after any stage 15
Stage 0: Pre-Work (read only, already done for you) 17
Stage 1: Reverse-engineer each application separately 18
Steps 18
Expected deliverables (per app) 18
What good output looks like (reference excerpt) 18
Review checklist 19
Stage 2: Semantic reconstruction (portfolio view) 20

Steps 20
Relationship vocabulary used 20
Expected deliverables 20
What good output looks like (reference excerpt) 20
Review checklist 21
Stage 3: Ground-truth evaluation 22
Steps 22
Scoring model 22
Expected deliverables 22
Reference result for comparison 23
Review checklist 23
Stage 4: Consolidation requirements and target design (planning only) 24
Steps 24
Expected deliverables 24
What good output looks like (reference excerpt) 24
Key reference decisions 25
Review checklist 25
Stage 5: Build the unified application 26
Steps 26
Expected deliverables 26
Code walkthrough: how the reference build preserved behavior 27
Review checklist 27
Module 4: Verify the modernized application end to end 29
4.1 Run it 29
4.2 Walk the cross-application workflows (now inside one app) 29
4.3 Check backward compatibility 29
4.4 Compare against the reference build 29
Applying the method to your own codebase 31
Prompt-writing patterns worth reusing 31
Troubleshooting 32
Appendix A: Repository map 33
Appendix B: Prompt to output cross-reference 33
Appendix C: Artifact conventions 34
Appendix D: Kiro quick reference for this exercise 34

About this tutorial

This tutorial walks you through a complete legacy modernization: three separate legacy applications (a Windows desktop app, a server-rendered web portal, and a REST API) all sharing one SQL Server database become a single modern ASP.NET Core web application. The journey runs in six stages. Each stage is driven by a carefully written prompt that you run in **Kiro**.

The repository you received contains everything you need:

- The **legacy application portfolio** (source code, database schema, stored procedures, seed data, tests).
- The **prompts** for every stage (`Prompts/`).
- A **reference run**: the artifacts and the final application produced when this method was first run. You will use it to compare against your own results.
- A **withheld answer key** (`benchmark/ground-truth/`) used to measure how accurately the AI reconstructed the legacy system.

What you will learn

- How to break a modernization into isolated, evidence-driven stages that an AI agent can run reliably.
- How to reverse-engineer business capabilities, entities, rules, workflows, and data dependencies from code, not from documentation or assumptions.
- How to measure the accuracy of AI-generated analysis against a ground truth before trusting it.
- How to turn verified findings into a consolidation specification with full functional traceability.
- How to have Kiro build, test, and validate the consolidated application against that specification.

What you will end up with

- Your own set of analysis, evaluation, and planning artifacts, generated by Kiro.
- A working **UnifiedRetail** ASP.NET Core (.NET 10) web application with customer, staff, and warehouse areas plus a backward-compatible API, running over the original database.
- A scorecard showing how accurate your Kiro run was, side by side with the reference run.

Time required

Plan for roughly **one to two working days** end to end. Stages 1 through 4 are analysis and planning and typically take tens of minutes each in Kiro. Stage 5 (the application build) is the longest stage and can take several hours of agent time. Actual times depend on the model you choose in Kiro and how much you review between steps.

> **IMPORTANT — Kiro is the operative tool for this exercise.** Every stage of this tutorial is executed by the Kiro AI agent. Kiro reads the code, runs the build and test commands, writes the artifacts, and implements the new application. Without Kiro you can still read the reference artifacts, but you cannot perform the exercise. Install and sign in to Kiro before you begin (see Module 1).

Kiro: the operative tool

What Kiro is

Kiro (https://kiro.dev) is an agentic AI development environment (IDE) from AWS. Besides editing code, Kiro's built-in AI agent can read an entire repository, reason across many files, run terminal commands (such as `dotnet build` and `dotnet test`), create and edit files, and iterate until a task is done. Kiro is available for Windows, macOS, and Linux.

Why this exercise depends on Kiro

Modernization need	What Kiro provides
Read and trace thousands of lines of C#, XAML, Razor, and SQL across many projects	The agent explores the workspace, follows call paths, and cites the files and symbols it used
Produce many structured artifacts (YAML catalogs, Markdown reports, Mermaid diagrams) per stage	The agent writes files directly into the workspace in the folder structure each prompt requires
Verify findings by running code	The agent runs builds, tests, and scripts in the integrated terminal (with your approval) and reports the real results
Build a full application incrementally	The agent implements milestone by milestone, builds and tests after each one, and fixes failures
Keep you in control	You approve commands, review diffs, and can roll back changes. Each stage runs in a fresh chat session you start

Kiro concepts used in this tutorial

This tutorial uses **Kiro IDE agent chat in Vibe mode**: you start a chat session, paste a stage prompt, and let the agent work. The concepts below are all you need. (Kiro's UI changes over time; if a label differs slightly in your version, look for the equivalent control.)

Concept	How it is used here
Chat panel	Where you paste each stage prompt and watch the agent work
Vibe session	A conversational agent session. Start a **new session for every stage** so earlier context does not leak into later stages
Autopilot / Supervised	Autopilot lets the agent make file changes without asking for each one; Supervised lets you review changes as they happen. Supervised is recommended for your first run of each stage
Command approval	Before the agent runs a terminal command, Kiro asks for approval. You can trust specific commands (for example `dotnet build`, `dotnet test`) to reduce prompts
Context with #	Typing `#` in chat attaches context, such as `#Folder` or `#File`. Use it to point the agent at the folders a stage is allowed to see
Model selection	Choose the model the agent uses. A strong reasoning model is recommended for these long, multi-file tasks
Checkpoints / revert	If a session goes wrong, revert the agent's changes and rerun. Git commits after every stage give you a second safety net

The scenario: a legacy retail portfolio

A fictional retailer runs three applications, built by different teams over several years. All three read from and write to one SQL Server database called `LegacyRetailDB`.

Application	Technology	Users	Responsibility	Source
OrderDesk	WPF desktop app (C#, XAML, MVVM), .NET 10	Sales and customer-service staff	Customers, sales quotes, discount approvals, staff orders, cancellations with override, return review, sales-agent tasks and commissions	`src/OrderDesk`
CustomerPortal	ASP.NET Core MVC with Razor views, .NET 10	Customers	Sign-in, profile, catalog, cart, coupons, idempotent checkout, order history, cancellation, returns, tracking, support tickets, wishlist	`src/CustomerPortal`
FulfillmentAPI	ASP.NET Core Web API with Swagger, .NET 10	Warehouse systems	Inventory allocation, picking, packaging, dispatch, carrier events, return receipt and restocking, warehouse reporting	`src/FulfillmentAPI`

The database (`database/`) has 40 tables, 5 stored procedures that hold real business rules, and a deterministic seed of more than 100 customers, 120 products, and 240 orders.

Why this portfolio is a realistic test

The portfolio was built on purpose to look like real enterprise software, with the kinds of problems that make modernization hard:

- Each app has **its own C# model of shared tables** (for example, three different `Order` classes over `dbo.Orders`).
- Some business rules are **duplicated** across apps; others are **implemented differently** on purpose (for example, staff and customers have different cancellation rules).
- Some rules live in **C# services**, others in **SQL stored procedures**.
- The apps never call each other. They are **coupled only through the shared database**.
- Similar names do not always mean the same thing (for example, `Customer` and `CustomerAccount` are different concepts).

A good modernization must discover all of this, keep the differences that matter to the business, and lose no functionality.

> **Note on language:** the legacy applications are written in C#. The same staged method applies unchanged to VB.NET, ASP.NET Web Forms, or other legacy stacks. See "Applying the method to your own codebase" at the end of this tutorial.

The target

One deployable **ASP.NET Core .NET 10 modular monolith**, called **UnifiedRetail**, that:

- Serves a **customer storefront**, a **staff area** (replacing the WPF desktop app), and a **warehouse area**.
- Preserves all 15 existing `/api/fulfillment/*` endpoints for backward compatibility.
- Implements all 47 discovered business capabilities.
- Runs on the **unchanged** database schema and stored procedures.

The six-stage method

Stage	Name	Goal	Prompt(s)	Output folder
0	Pre-Work	Build the legacy portfolio and the ground-truth answer key (already done for you)	`01.CreateDemoApps.md`, `02.GroundTruthBuilder..md`	`src/`, `database/`, `benchmark/ground-truth/`
1	Reverse-engineer each app separately	Rebuild each app's capabilities, entities, rules, workflows, and DB dependencies from code only	`03.OrderDesk.md`, `04.CustomerPortal.md`, `05.FullfillmentAPI.md`	`analysis-output/<app>/`
2	Semantic reconstruction	Merge the three analyses into one portfolio view, then verify every relationship against source	`06. SementicReconsutuction.md`	`analysis-output/portfolio/`
3	Ground-truth evaluation	Score the reconstruction against the withheld answer key with deterministic metrics	`07.GroundTruth.md`	`evaluation-output/`
4	Consolidation requirements and target design	Turn verified findings into an approved specification. No code yet	`08.ConsolidationRequirmentsAndTargetDesign.md` + `07.5GroundTruth2.md` (add-on)	`consolidation-planning/`
5	Application build	Implement, test, and validate the unified application	`09.BuildTheApp.md`	`consolidated-app/`

Principles that make the method work

1. **Evidence over assumption.** Every finding must cite a real file, class, method, or SQL object. Findings are labelled `static_verified` (source inspected), `behavior_verified` (test executed), or `inferred`.
2. **Isolation before integration.** Each application is analyzed alone first, so the agent cannot "guess" from the other apps. Cross-application conclusions come only in Stage 2.

3. **Measure before you trust.** Stage 3 grades the AI's analysis against a ground truth before anything is built on it.
4. **Nothing is silently discarded.** Every capability gets an explicit disposition and a parity test in Stage 4.
5. **Preserve legitimate differences.** Different rules for different actors are kept as explicit policies, not "fixed".
6. **Human gates.** You review and approve the output of every stage before starting the next.

Module 1: Set up your environment

1.1 Prerequisites

Requirement	Why	Check
Kiro IDE, installed and signed in	Runs every stage of the exercise	Open Kiro and confirm the chat panel is available
Windows 10/11	The OrderDesk WPF app only builds and runs on Windows. Everything else is cross-platform	n/a
.NET 10 SDK	Build and run all applications	`dotnet --version` shows 10.x
SQL Server: LocalDB (default), a local instance, or a container	The shared `LegacyRetailDB` database	`sqllocaldb info` lists `MSSQLLocalDB`
Git	Branching, committing each stage, comparing with the reference run	`git --version`
Python 3 (PyYAML optional)	Stage 3 deterministic scorer and YAML validation	`python --version`
PowerShell	Startup scripts in `scripts/`	Built into Windows

1.2 Install and sign in to Kiro

1. Download Kiro from **https://kiro.dev** and install it.
2. Start Kiro and sign in with one of the supported methods (for example GitHub, Google, AWS Builder ID, or your organization's AWS IAM Identity Center).
3. Open the chat panel and confirm you can start a new session.
4. In the model selector, choose a strong reasoning model. Stages 1 to 5 are long, multi-file tasks.

> **Tip:** Kiro plans include usage limits. Stage 5 in particular uses a lot of agent time, so check your plan's remaining usage before you start it.

1.3 Get the repository and build it

Clone or unzip the repository, then open a terminal in its root folder:

```powershell
git clone <repository-url> retail-portfolio-modernization
cd retail-portfolio-modernization

# Build all legacy projects (expect: Build succeeded, 0 errors)
dotnet build LegacyRetailPortfolio.slnx
```

1.4 Provision the database

The provisioner creates `LegacyRetailDB`, its 40 tables and 5 stored procedures, and deterministic seed data. It is safe to rerun.

```powershell
```

```
./scripts/provision-db.ps1            # add -Reset to drop and rebuild
```

> **Warning:** Only point the provisioner at a disposable, local database. `-Reset` drops and recreates `LegacyRetailDB`.

To use a SQL Server container or another instance instead of LocalDB, set both connection strings first:

```powershell
$env:LEGACYRETAIL_CONNECTION = "Server=localhost,1433;Database=LegacyRetailDB;User
Id=sa;Password=Your_strong_P@ss;TrustServerCertificate=True;"
$env:LEGACYRETAIL_MASTER_CONNECTION = "Server=localhost,1433;Database=master;User
Id=sa;Password=Your_strong_P@ss;TrustServerCertificate=True;"
```

1.5 Run the legacy applications

Spend 15 minutes using the legacy apps. You will understand the later stages much better.

```powershell
./scripts/run-all.ps1        # starts all three apps, each in its own window
```

App	How to reach it	Sign-in
CustomerPortal	http://localhost:5101	`user1` to `user80`, password `Password123!`
FulfillmentAPI	http://localhost:5201/swagger	Header `X-Api-Key: dev-fulfillment-key`
OrderDesk	Desktop window	No login; choose the acting agent in the UI

Try sample workflow **B**: sign in to the portal, add products to the cart, apply a coupon, and check out. Then look up the order through the API's `GET /api/fulfillment/orders/pending` endpoint in Swagger.

1.6 Run the legacy tests

```powershell
dotnet test LegacyRetailPortfolio.slnx
```

The unit-test projects (`OrderDesk.Tests`, `CustomerPortal.Tests`, `FulfillmentAPI.Tests`) need no database. `IntegrationTests` needs the provisioned database.

1.7 Open the repository in Kiro

In Kiro, choose **File > Open Folder** and open the repository root. Let Kiro index the workspace. When prompted, confirm that you trust the folder.

Module 2: Prepare a clean learner workspace

The repository already contains the reference run's outputs, the answer key, and the original design specification. If Kiro can see them while it works, it can copy answers instead of discovering them, and your experiment would be meaningless. In this module you will:

1. Keep a **read-only reference copy** of everything, outside your Kiro workspace.
2. Create a **learner branch** where the generated outputs, answer key, and design-spec prompts are removed.

2.1 Create your learner branch and the reference copy

From the repository root:

```powershell
# 1. Your working branch
git checkout -b my-modernization-run

# 2. A second, read-only checkout of 'main' next to your workspace.
#    Do NOT open this folder in Kiro.
git worktree add ../modernization-reference main
```

You now have `../modernization-reference/` containing the original prompts, reference outputs, and ground truth. You will copy prompts from it and compare your results against it.

2.2 Strip the learner branch down to the legacy code

```powershell
# Remove the reference outputs (Kiro will regenerate them)
git rm -r -q analysis-output evaluation-output consolidation-planning consolidated-app

# Withhold the answer key, the design spec, and the reference narrative from the agent
git rm -r -q benchmark Prompts ModernizationStep.md docs/tutorial

git commit -m "Learner run: start from legacy code only"
```

> **Why also remove the prompts and this tutorial?** The Stage 0 prompt (`01.CreateDemoApps.md`) is the original design specification and lists the deliberate overlaps and traps. This tutorial quotes reference results. Kiro must not read either during Stages 1 and 2. You will paste each stage prompt from `../modernization-reference/Prompts/` instead.

Your workspace now contains only what a real modernization team would have: `src/`, `database/`, `tests/`, `docs/`, `scripts/`, the solution file, and `README.md`.

> **Tip:** `docs/IMPLEMENTATION-PROGRESS.md` and `docs/ARCHITECTURE.md` are ordinary project documentation, like what a real legacy system might have. They are allowed inputs.

2.3 Optional: give Kiro trusted commands

To avoid approving the same safe commands over and over, trust read-only and build/test commands when Kiro first asks (for example `dotnet build`, `dotnet test`, `git status`, `git diff`). Keep approval on for anything that deletes files, resets the database, or touches anything outside the workspace.

Module 3: How to run a stage in Kiro (repeat for every stage)

Every stage follows the same eight-step loop. Learn it once and then apply it to each stage.

1. **Start a new Vibe session** in the Kiro chat panel. Never continue a previous stage's session, because it carries context that could break isolation.
2. **Open the stage prompt** from `../modernization-reference/Prompts/<stage folder>/<file>.md` in any text editor and copy its full contents.
3. **Paste the prompt** into the Kiro chat. Optionally attach the allowed inputs with `#Folder` (for example `#Folder src/OrderDesk`), to reinforce the scope.
4. **Choose Supervised** for your first run of a stage (to review changes), or **Autopilot** once you trust the flow.
5. **Approve commands** as Kiro asks to run builds, tests, or scripts. Read each command before approving it. Decline anything that resets the database or touches folders outside the stage's scope.
6. **Let the agent finish.** If it stops early, reply with `Continue. Complete all remaining deliverables listed in the prompt.`
7. **Review the output** using the stage's checklist in this tutorial, then commit:

```powershell
git add -A
git commit -m "Stage N: <description>"
```

8. **Compare with the reference run** (see 3.1).

3.1 Comparing your results with the reference

Because your branch started from `main`, git can compare any folder of your run with the reference:

```powershell
# Which files differ, and how much
git diff main --stat -- analysis-output/orderdesk

# Full diff of one artifact
git diff main -- analysis-output/orderdesk/capabilities.yaml
```

You can also open both versions side by side in Kiro's editor: your file in the workspace, and the reference in `../modernization-reference/`.

> **Do not expect identical text.** AI output varies between runs. Compare **substance**: did your run find the same capabilities, rules, entities, and traps? Did it cite real evidence? Are the IDs consistent across files?

3.2 Quick validity checks you can run after any stage

```powershell
# Every YAML file in the stage's output folder parses (requires: pip install pyyaml)
python ../modernization-reference/docs/tutorial/tools/validate_yaml.py analysis-output
```

You can also ask Kiro in the same session: *"Validate that every YAML file you produced parses with a strict YAML parser, and that every source path you cited exists. Fix any failures."*

> **Why this matters:** the most common slip is an unquoted `?` inside a YAML list, such as `decision_points:`

> `[locked?, password valid?]`. Even the reference run has a few files with this problem (for example `analysis-output/*/workflows.yaml`). Ask Kiro to quote such values.

Stage 0: Pre-Work (read only, already done for you)

Goal: create the experiment: a realistic legacy portfolio plus an answer key to grade the AI later.

Prompts:

- `Prompts/Stage 0. PreWork/01.CreateDemoApps.md` is the master build prompt for the three apps and the shared database. It specifies the target mix of app-specific (50 to 60%), partially overlapping (20 to 30%), and broadly shared (15 to 20%) capabilities, plus deliberate "imperfections" such as duplicated rules, two C# models over one table, and a bounded cancellation difference.
- `Prompts/Stage 0. PreWork/02.GroundTruthBuilder..md` is the agent prompt that built the evidence-backed answer key in `benchmark/ground-truth/`.

What to do: read both prompts in `../modernization-reference/Prompts/Stage 0. PreWork/` so you understand how the benchmark was built. **Do not run them**: the portfolio already exists, and rebuilding it would change the code that the answer key describes.

> **Why this stage matters for your own projects:** on a real codebase, you will not have a ground truth. You can still create a small one: ask subject-matter experts to write down 10 to 20 capabilities and rules they know exist, with code locations, and keep them away from the agent. That gives you a way to measure your Stage 1 and 2 accuracy.

Stage 1: Reverse-engineer each application separately

Goal: for each application, independently reconstruct what it does from implementation evidence only: capabilities, entities, business rules, workflows, and database dependencies.

Run three separate Kiro sessions, one per application:

Session	Prompt	Allowed inputs	Output folder	ID prefix
1a	`Stage 1. ReverseEngineer-Seperately/03.OrderDesk.md`	`src/OrderDesk/`, `src/Shared/`, `database/`, `tests/OrderDesk.Tests/`	`analysis-output/orderdesk/`	`OD-`
1b	`Stage 1. ReverseEngineer-Seperately/04.CustomerPortal.md`	`src/CustomerPortal/`, `src/Shared/`, `database/`, `tests/CustomerPortal.Tests/`	`analysis-output/customerportal/`	`CP-`
1c	`Stage 1. ReverseEngineer-Seperately/05.FullfillmentAPI.md`	`src/FulfillmentAPI/`, `src/Shared/`, `database/`, `tests/FulfillmentAPI.Tests/`	`analysis-output/fulfillmentapi/`	`FA-`

Steps

1. Start a **new Vibe session** in Kiro.
2. Paste the contents of `03.OrderDesk.md`.
3. Optionally attach `#Folder src/OrderDesk` and `#Folder database` and add one line at the end: *"Do not open any files under src/CustomerPortal or src/FulfillmentAPI."*
4. Let Kiro run. It will read the projects, trace WPF views to ViewModels to services to EF Core and stored procedures, and may run the unit tests to upgrade findings to `behavior_verified`.
5. Review, commit (`Stage 1a: OrderDesk analysis`), and compare.
6. Repeat in **new sessions** for CustomerPortal (1b) and FulfillmentAPI (1c).

Expected deliverables (per app)

```text
analysis-output/<app>/
├── application-profile.md   # technical + business profile
├── capabilities.yaml        # capability catalog grouped into domains
├── entities.yaml            # domain entities, table mappings, ownership
├── business-rules.yaml      # rules with predicate, location, evidence
├── workflows.yaml           # end-to-end workflows (+ Mermaid diagrams)
├── dependencies.yaml        # tables / procs read, written, executed
├── evidence-index.yaml      # every cited source location
├── analysis-gaps.md         # what could not be determined
├── summary.md               # one-page summary
└── state-machines.md        # FulfillmentAPI only
```

What good output looks like (reference excerpt)

From the reference `analysis-output/orderdesk/capabilities.yaml`:

```yaml
domains:
  - id: OD-DOM-QUOTING
    name: Sales Quotation & Discount Approval
```

```
    capabilities: [OD-CAP-005, OD-CAP-006, OD-CAP-007, OD-CAP-008, OD-CAP-009]

capabilities:
  - id: OD-CAP-001
    domain: OD-DOM-CUSTOMER
    name: Search customers
    description: Staff search customers by first name, last name, or email (capped at 200
results).
    actor: Sales / customer-service staff
    trigger: Customers tab "Search" command
    inputs: [search term (optional)]
    outputs: [list of Customer]
    entities: [OD-ENT-001]
```

Review checklist

- ☐ All 9 files (10 for FulfillmentAPI) exist and every YAML file parses.
- ☐ IDs are stable and cross-referenced (a capability lists its rule IDs, and those rules exist).
- ☐ Evidence paths point at real files and symbols. Spot-check five of them.
- ☐ Stored procedures are analyzed by their **body**, not just their name. For example, OrderDesk's staff cancellation rule lives in `usp_EvaluateCancellationEligibility` in `database/procedures/procedures.sql`.
- ☐ Nothing is claimed as `behavior_verified` unless a test actually ran in the session.
- ☐ The analysis does **not** compare the app with the other apps (that is Stage 2's job).

Reference result: 16 OrderDesk, 18 CustomerPortal, and 13 FulfillmentAPI capabilities (47 in total) and 67 business rules.

Stage 2: Semantic reconstruction (portfolio view)

Goal: combine the three independent analyses into one portfolio model: canonical capabilities, how capabilities relate across apps, canonical domains and entities, cross-app dependencies and workflows. Then verify every proposed relationship against the source.

Prompt: `Stage 2. Semenatic Reconstruction/06. SementicReconsutuction.md`

Steps

1. Start a **new Vibe session**.
2. Paste the prompt. It runs in two phases:
   - **Phase A, Semantic consolidation:** uses only `analysis-output/{orderdesk,customerportal,fulfillmentapi}/`.
   - **Phase B, Source-level verification:** inspects all three codebases, the SQL, and tests to confirm, revise, or reject each Phase A relationship.
3. If Kiro tries to go straight to source code, remind it: *"Complete Phase A from the Stage 1 outputs only and save the candidate findings before starting Phase B."*
4. Review, commit (`Stage 2: portfolio reconstruction`), and compare.

Relationship vocabulary used

Class	Meaning
`duplicate`	Substantially equivalent functionality
`partial_overlap`	Same capability, different responsibilities, scope, or rules
`complementary`	Different capabilities that contribute to one business process
`shared_data_only`	Shared data, no significant functional overlap
`unique`	Belongs to one application

Expected deliverables

```text
analysis-output/portfolio/
├── portfolio-capabilities.yaml
├── capability-overlaps.yaml
├── canonical-domains.yaml
├── cross-app-dependencies.yaml
├── cross-app-workflows.yaml
├── business-rule-comparison.yaml
├── portfolio-architecture.md     # capability map, matrices, Mermaid diagrams
├── evidence-index.yaml
└── unresolved-findings.md
```

What good output looks like (reference excerpt)

From the reference `analysis-output/portfolio/capability-overlaps.yaml`:

```yaml
  - id: OVL-CUSTOMER
    capability: PF-CAP-CUSTOMER-MANAGE
    relationship: partial_overlap
```

```
    apps: [APP-ORDERDESK, APP-PORTAL, APP-FULFILL]
    rationale: >
      OrderDesk edits the full contact/shipping record; CustomerPortal edits only a
      self-service subset (phone + shipping); FulfillmentAPI only counts distinct
      customers inside a report. Same physical table (dbo.Customers) via different
      C# models; not duplicate functionality.
    verification: static_verified
    evidence:
      source:
        - src/OrderDesk/OrderDesk.Application/CustomerService.cs#UpdateContactAsync
        - src/CustomerPortal/CustomerPortal.Application/ProfileService.cs#UpdateSelfServiceAsync
        - database/procedures/procedures.sql#usp_WarehouseActivitySummary
```

Review checklist

- ☐ Original Stage 1 IDs are preserved for traceability.
- ☐ No relationship is marked `duplicate` just because two apps use the same table or class name.
- ☐ Shared tables with several C# models are identified (for example, three `Order` models over `dbo.Orders`).
- ☐ `Customer` and `CustomerAccount` are kept as **distinct** concepts.
- ☐ The staff vs customer **cancellation difference** is recorded as a legitimate, bounded divergence.

Customer rule, in C# (`src/CustomerPortal/CustomerPortal.Domain/PortalRules.cs`):

```csharp
public static bool CustomerCanCancel(string orderStatus) =>
    orderStatus is "Pending" or "Confirmed";
```

Staff rule, in SQL (`usp_EvaluateCancellationEligibility`):

```sql
ELSE IF @Status IN ('Pending','Confirmed','Allocated')
    SET @IsEligible = 1; SET @Reason = 'Eligible for staff cancellation.';
```

- ☐ Coupling is described as **data-mediated** through the shared database. There are no invented API calls between apps.
- ☐ Phase A hypotheses that could not be verified are listed in `unresolved-findings.md`, not silently promoted.

Stage 3: Ground-truth evaluation

Goal: measure how accurate Stages 1 and 2 were, using the withheld answer key and deterministic, code-computed metrics, **before** anything is built on top of them.

Prompt: `Stage 3. GroundTruth/07.GroundTruth.md`

Steps

1. **Reveal the answer key** to the workspace (for this stage only):

```powershell
Copy-Item -Recurse ../modernization-reference/benchmark ./benchmark
```

2. Start a **new Vibe session** and paste `07.GroundTruth.md`. Kiro acts as an independent evaluator: it matches every ground-truth item to a prediction (`exact`, `partial`, `incorrect`, unmatched, or `ambiguous`), freezes those decisions in YAML match tables, and writes `score.py` to compute the metrics.
3. When Kiro asks to run the scorer, approve it. You can also run it yourself:

```powershell
python evaluation-output/score.py
```

4. Review the results, then **hide the answer key again** before Stage 4 so planning is based on your findings, not the answers:

```powershell
Remove-Item -Recurse -Force ./benchmark
git add -A
git commit -m "Stage 3: ground-truth evaluation"
```

Scoring model

Category	Weight	Unit
Business capability discovery	25%	ground-truth capability (27)
Domain and entity discovery	20%	physical table (40) + multi-model traps
Overlap classification	20%	overlap scenario (6)
Business-rule discovery	15%	ground-truth rule (19)
Cross-app dependencies	10%	dependency edges and stored procedures
Evidence accuracy	10%	sampled evidence references
Workflows	diagnostic	cross-app workflows

Two metric families are reported: **strict** (only `exact` counts) and **partial credit** (`exact` = 1.0, `partial` = 0.5).

Expected deliverables

`evaluation-output/` with `evaluation-summary.md`, `evaluation-methodology.md`, `scorecard.yaml`, `score.py`, `confusion-matrix.csv`, one match file per category, `stage-`

`comparison.md`, `false-positives.md`, `false-negatives.md`, `adjudication-queue.yaml`, and `evaluation-dashboard.html`.

Reference result for comparison

Category	Reference score
Capability discovery	98.0% (recall 100%)
Domain and entity	100%
Overlap classification	82.5%
Business rules	100% (19/19 exact)
Dependencies	100%
Evidence accuracy	95.5%
Weighted overall	**95.5%**

The reference run's main weakness was **overlap classification**: two "broadly shared" scenarios were labeled `partial_overlap`, because the prediction vocabulary has no "broadly shared" tier. Open `evaluation-dashboard.html` in a browser for a visual summary.

Review checklist

- ☐ `score.py` runs cleanly and regenerates `scorecard.yaml` and `confusion-matrix.csv`.
- ☐ Every ground-truth item has exactly one classification.
- ☐ Ambiguous matches are in `adjudication-queue.yaml` with both interpretations, not silently resolved.
- ☐ You have read `false-negatives.md` and `false-positives.md`. These are the gaps Stage 4 must handle.

> **Tip:** if your score is much lower than the reference, open `stage-comparison.md` to see whether findings were missed in Stage 1 (discovery) or lost in Stage 2 (consolidation). Rerunning only the weak Stage 1 session is often enough.

Stage 4: Consolidation requirements and target design (planning only)

Goal: translate the verified findings into a complete, evidence-backed consolidation specification for **one ASP.NET Core web application with a backward-compatible API**. No code is written in this stage.

Prompts:

- Main prompt: `Stage 4. Consolidation Requirments/08.ConsolidationRequirmentsAndTargetDesign.md`
- Add-on: `Stage 3. GroundTruth/07.5GroundTruth2.md`. Despite its folder, this file contains **"Stage 3-informed" additional requirements for Stage 4**: resolve uncertain overlap classifications, require full traceability, preserve context-specific policies, define API compatibility, and pass a consolidation readiness gate.

Steps

1. Start a **new Vibe session**.
2. Paste the full contents of `08.ConsolidationRequirmentsAndTargetDesign.md`, then **append** the full contents of `07.5GroundTruth2.md` in the same message.
3. If your own Stage 3 scores differ from the reference, edit the score values at the top of the add-on to match your `scorecard.yaml` before sending.
4. Let Kiro produce the planning set. It will go back to source code and SQL to verify details.
5. **Human approval gate:** read `consolidation-decisions.md` and `consolidation-readiness-report.md`. Approve or adjust the decisions before Stage 5. If you change a decision, tell Kiro in the same session and ask it to update every affected artifact.
6. Commit (`Stage 4: consolidation plan`) and compare.

Expected deliverables

```text
consolidation-planning/
├── unified-capability-catalog.yaml     # every capability with a disposition
├── capability-reconciliation.yaml      # overlap-by-overlap decisions
├── consolidated-domain-model.md
├── module-ownership-matrix.yaml        # logical modules and ownership
├── unified-web-requirements.md         # customer / staff / warehouse journeys
├── api-compatibility-inventory.yaml    # contract for every existing endpoint
├── data-consolidation-assessment.md    # schema, procs, migration risks
├── functional-parity-matrix.yaml       # capability -> acceptance -> tests
├── architecture-options.md             # options + trade-offs
├── consolidation-decisions.md
├── consolidation-readiness-report.md   # readiness gate (the add-on may also produce a
checklist)
```

What good output looks like (reference excerpt)

From the reference `consolidation-planning/functional-parity-matrix.yaml`:

```yaml
  - cap: OD-CAP-002
    target_feature: Staff full customer edit
    target_module: MOD-CUSTOMER
    required_behavior: Edit all contact+shipping; email required+unique.
    acceptance: [Blank email rejected; duplicate email rejected (UQ_Customers_Email); all fields
```

```
persist.]
    existing_tests: []
    new_parity_tests: [Staff_edit_persists_all_fields; Email_required_and_unique]
    migration_risk: low
```

Key reference decisions

- **Modular monolith**, one ASP.NET Core host. No microservices.
- **13 logical modules** (logical boundaries, not necessarily 13 projects).
- **Keep the database schema and all 5 stored procedures unchanged.**
- **Preserve the `/api/fulfillment/ contract** (routes, verbs, payloads, status codes,* X-Api-Key` behavior, idempotency).
- **Preserve per-actor differences** (for example, staff vs customer cancellation rules).
- **Every one of the 47 capabilities dispositioned; none discarded.**

Review checklist

- ☐ Every Stage 1 capability ID appears in `unified-capability-catalog.yaml` and `functional-parity-matrix.yaml`.
- ☐ Every overlap has a decision: unify, keep context-specific, or escalate to the business.
- ☐ Every existing API endpoint has a compatibility strategy and contract tests listed.
- ☐ Verified behavior, proposed behavior, business decisions, assumptions, and gaps are clearly separated.
- ☐ The readiness report shows the gate as passed, or lists exactly what fails.

Stage 5: Build the unified application

Goal: implement, test, and validate the consolidated **UnifiedRetail** application from the approved Stage 4 plan.

Prompt: `Stage 5. ApplicationBuild/09.BuildTheApp.md`

Steps

1. Make sure the database is provisioned (Module 1.4). The build stage runs integration tests against it.
2. Start a **new Vibe session** and paste `09.BuildTheApp.md`.
3. Use **Supervised** mode for Milestone 1 so you can see the project structure being created, then switch to **Autopilot** if you are comfortable.
4. Approve `dotnet build`, `dotnet test`, and `dotnet run` commands as they come up. **Decline** any command that would reset or reseed the database unless you have confirmed it is your disposable local instance.
5. Kiro works through five milestones, building and testing after each one:

Milestone	Scope
M1 Foundation	Single host, persistence, DI, cookie auth, role-based authorization, module structure
M2 API compatibility	Thin `/api/fulfillment/*` controllers over the fulfillment services, plus contract tests
M3 Customer web	Storefront: auth, catalog, cart, coupons, checkout, orders, returns, tracking, support, wishlist
M4 Staff web	Replaces the WPF app: customers, quotes, discount approvals, orders and overrides, returns review, agents
M5 Warehouse web + validation	Warehouse UI over the same services as the API; cross-module workflow tests; validation report

6. **Long sessions:** if the session ends or the context fills up before all milestones are done, start a **new session** and send:

   "Continue the Stage 5 build. Read consolidated-app/implementation-progress.md and the Stage 4 artifacts in consolidation-planning/, then resume from the first incomplete milestone. Follow the original rules: build and test after each milestone, and do not claim tests passed without running them."

7. Commit after each milestone (`Stage 5 M1: foundation`, and so on).

Expected deliverables

```
TEXT
consolidated-app/
├── src/UnifiedRetail/                  # single ASP.NET Core host
│   ├── Program.cs                      # DI, auth, middleware, routing
│   ├── Platform/                       # API-key + exception middleware, current user
│   ├── Modules/Fulfillment/Api/        # preserved /api/fulfillment/* controllers
│   ├── Controllers/ + Views/           # customer storefront
│   └── Areas/Staff, Areas/Warehouse    # staff and warehouse web UIs
├── tests/UnifiedRetail.Tests/          # integration, contract, workflow tests
├── README.md
├── implementation-progress.md
└── validation-report.md
```

Code walkthrough: how the reference build preserved behavior

1. Reuse proven business logic instead of rewriting it. The reference host references the existing domain, application, and infrastructure projects and registers their services. The real rules, EF mappings, and stored-procedure calls therefore run unchanged (`consolidated-app/src/UnifiedRetail/Program.cs`):

```csharp
// Persistence (provisional PoC decision): reuse the three proven per-app DbContexts
// over the ONE shared LegacyRetailDB connection.
builder.Services.AddDbContextFactory<OrderDeskDbContext>(o => o.UseSqlServer(connectionString));
builder.Services.AddDbContext<PortalDbContext>(o => o.UseSqlServer(connectionString));
builder.Services.AddDbContext<FulfillmentDbContext>(o => o.UseSqlServer(connectionString));

// Application services reused verbatim from the three source apps
builder.Services.AddScoped<OrderDesk.Application.QuoteService>();
builder.Services.AddScoped<CustomerPortal.Application.CheckoutService>();
builder.Services.AddScoped<FulfillmentAPI.Application.AllocationService>();
// ...
```

2. Thin compatibility controllers: one business core, two surfaces. The preserved API and the new warehouse UI call the **same** application services (`Modules/Fulfillment/Api/CompatibilityControllers.cs`):

```csharp
[ApiController]
[Route("api/fulfillment/orders")]
public class OrdersController : FulfillmentControllerBase
{
    private readonly FulfillmentQueryService _query;
    public OrdersController(FulfillmentQueryService query) => _query = query;

    [HttpGet("pending")]
    public async Task<IActionResult> GetPending(CancellationToken ct)
        => Ok(await _query.GetPendingOrdersAsync(ct));

    [HttpGet("{orderId:int}")]
    public async Task<IActionResult> Get(int orderId, CancellationToken ct)
    {
        var o = await _query.GetOrderAsync(orderId, ct);
        return o is null ? NotFound(new { error = "Order not found." }) : Ok(o);
    }
}
```

3. Desktop to web. The WPF OrderDesk screens (customers, quotes, approvals, orders, returns, agents) become the Razor views under `Areas/Staff/Views/`, backed by the same OrderDesk application services the desktop ViewModels used.

Review checklist

- ☐ `dotnet build consolidated-app/src/UnifiedRetail/UnifiedRetail.csproj` succeeds.
- ☐ `dotnet test consolidated-app/tests/UnifiedRetail.Tests` passes. Any failures or skips are explained in `validation-report.md`.
- ☐ `validation-report.md` covers **all 47 capabilities** and **all 15 API endpoints**.
- ☐ The original three apps and the Stage 4 documents were not modified (`git diff main --stat -- src consolidation-planning`).
- ☐ No placeholder screens: every page you click performs a real operation on the database.

Reference result: builds clean; **27 tests passed, 0 failed, 2 skipped** (a documented test-harness cold-start timing issue; the endpoint itself is verified); all 47 capabilities implemented; all 15 endpoints preserved; schema and procedures unchanged.

Module 4: Verify the modernized application end to end

4.1 Run it

```powershell
dotnet run --project consolidated-app/src/UnifiedRetail/UnifiedRetail.csproj --launch-profile http
```

Browse to **http://localhost:5280**.

Area	How to sign in
Customer storefront	`/Account/Login` with `user1` to `user80`, password `Password123!`
Staff area (replaces OrderDesk)	`/Account/Staff` role picker (proof-of-concept sign-in; the legacy desktop app had no login)
Warehouse area	`/Account/Staff`, choose a warehouse role
Preserved API	`/api/fulfillment/*` with header `X-Api-Key: dev-fulfillment-key`; Swagger at `/swagger`

4.2 Walk the cross-application workflows (now inside one app)

Workflow	Steps in UnifiedRetail
A: Staff order	Staff: create quote > add lines > apply discount above threshold > approve in Approvals > convert to order. Warehouse: allocate > pick > package > dispatch
B: Customer checkout	Customer: add to cart > apply coupon > checkout. Warehouse: fulfill. Customer: see tracking on the order page
C: Return	Customer: request return on a delivered order. Staff: approve in Returns. Warehouse: receive and restock
D: Cancellation	Customer: cancel a Pending order (allowed); try an Allocated order (refused). Staff: cancel an Allocated order (allowed) or override with authorization
E: Warehouse	Pending orders > allocate > pick list > package > dispatch > carrier events > tracking

4.3 Check backward compatibility

```powershell
curl -H "X-Api-Key: dev-fulfillment-key" http://localhost:5280/api/fulfillment/orders/pending
curl -i http://localhost:5280/api/fulfillment/orders/pending      # expect 401 without the key
```

4.4 Compare against the reference build

```powershell
git diff main --stat -- consolidated-app
```

Your implementation will differ in detail. What matters is that your `validation-report.md` shows the same coverage (47 capabilities, 15 endpoints) and that your tests pass.

Applying the method to your own codebase

The prompts are written for this portfolio, but the method is general. To use it on your own legacy system (C#, **VB.NET**, ASP.NET Web Forms, WinForms, Java, COBOL front-ends with SQL back-ends, and so on):

1. **Inventory the applications** that share data or business processes. Treat each as a Stage 1 unit.
2. **Adapt the Stage 1 prompt per app:** change the role line (for example, *"specializing in VB.NET WinForms and ADO.NET"*), the allowed paths, the technical discovery list (for example code-behind, `DataSet`s, `SqlCommand`, `Handles` clauses), the output folder, and the ID prefix.
3. **Build a small ground truth** with subject-matter experts (see Stage 0) and withhold it from Kiro, so you can run Stage 3 and know how far to trust the analysis.
4. **Keep Stage 2's two phases.** Consolidating from analyses first and then verifying against source is what prevents invented relationships.
5. **Change the target in Stage 4** to your chosen architecture, keeping the rules about traceability, parity, API compatibility, and the readiness gate.
6. **Keep Stage 5 incremental**, with build and test after every milestone, and require a validation report backed by executed tests.
7. **Keep humans at the gates.** Review and commit after every stage.

Prompt-writing patterns worth reusing

Pattern	Example from the prompts
Explicit scope and isolation	*"Analyze only... Do not analyze any other application."*
Evidence requirement	*"Every substantive business finding must contain a source reference."*
Honest verification labels	`static_verified` / `behavior_verified` / `inferred`
Anti-shortcut rules	*"Do not equate C# classes or UI screens directly with business capabilities."*
Fixed deliverables and stable IDs	`OD-CAP-001`, `CP-RULE-001`, `FA-WF-001`
No fabricated results	*"Do not claim tests passed without executing them."*
Plan before build	*"Produce the planning artifacts before generating implementation code."*

Troubleshooting

Symptom	Likely cause	Fix
Kiro reads files outside the stage scope	The prompt alone was not enough	Start a new session; attach only the allowed `#Folder` contexts; add *"Do not open files under ..."* to the prompt
Kiro stops before producing all deliverables	Long task or context limit	Reply *"Continue and complete all remaining deliverables."* For Stage 5, use the resume message from Stage 5, step 6
Findings claimed as `behavior_verified` without tests	The agent over-claimed	Ask Kiro to downgrade any finding not backed by a test run in the session
`Cannot open database "LegacyRetailDB"`	Database not provisioned or wrong connection	Run `./scripts/provision-db.ps1`; check `LEGACYRETAIL_CONNECTION`
OrderDesk will not build	Not on Windows	WPF requires Windows; the other projects build anywhere
Integration tests fail but unit tests pass	Database missing or stale	`./scripts/provision-db.ps1 -Reset` (disposable local DB only)
Port 5280 / 5101 / 5201 already in use	Another app instance is running	Stop the other process or change `Properties/launchSettings.json`
`score.py` errors on import	PyYAML missing	`pip install pyyaml`, or rely on the script's embedded fallback tables
Your Stage 3 score is much lower than the reference	Stage 1 missed findings, or Stage 2 merged them away	Read `stage-comparison.md` and `false-negatives.md`; rerun the weak Stage 1 session in a new Kiro session

Appendix A: Repository map

Path	Contents
src/OrderDesk, src/CustomerPortal, src/FulfillmentAPI	Legacy apps (Domain / Application / Infrastructure / UI projects each)
src/Shared/LegacyRetail.Common	Shared infrastructure only (connection config, PBKDF2 hashing). No business logic
database/schema, database/procedures	DDL (40 tables) and the 5 stored procedures
database/provisioning/DbProvisioner	Creates, migrates, and seeds LegacyRetailDB
tests/	Per-app unit tests and live-database integration tests
scripts/	Provisioning and run scripts
docs/	Legacy architecture docs (Mermaid)
benchmark/ground-truth/	Answer key. **Withhold from Kiro except in Stage 3**
Prompts/	Stage 0 to 5 prompts
analysis-output/	Reference Stage 1 and Stage 2 outputs
evaluation-output/	Reference Stage 3 outputs
consolidation-planning/	Reference Stage 4 outputs
consolidated-app/	Reference Stage 5 unified application
ModernizationStep.md	Narrative of the reference run

Appendix B: Prompt to output cross-reference

Stage	Prompt file	Kiro session(s)	Output
0	Stage 0. PreWork/01.CreateDemoApps.md, 02.GroundTruthBuilder..md	Not run (read only)	src/, database/, benchmark/ground-truth/
1	Stage 1. ReverseEngineer-Seperately/03.OrderDesk.md	1	analysis-output/orderdesk/
1	Stage 1. ReverseEngineer-Seperately/04.CustomerPortal.md	1	analysis-output/customerportal/
1	Stage 1. ReverseEngineer-Seperately/05.FullfillmentAPI.md	1	analysis-output/fulfillmentapi/
2	Stage 2. Semenatic Reconstruction/06. SementicReconsutuction.md	1	analysis-output/portfolio/
3	Stage 3. GroundTruth/07.GroundTruth.md	1	evaluation-output/
4	Stage 4. Consolidation Requirments/08...md + Stage 3. GroundTruth/07.5GroundTruth2.md	1	consolidation-planning/
5	Stage 5. ApplicationBuild/09.BuildTheApp.md	1 or more (resume as needed)	consolidated-app/

Appendix C: Artifact conventions

Convention	Values
Stage 1 ID prefixes	`OD-` OrderDesk, `CP-` CustomerPortal, `FA-` FulfillmentAPI
ID types	`-CAP-` capability, `-ENT-` entity, `-RULE-` business rule, `-WF-` workflow, `-DOM-` domain
Stage 2 IDs	`PF-CAP-*` portfolio capability, `OVL-*` overlap, `XWF-*` cross-app workflow
Confidence	`high`, `medium`, `low`
Verification	`static_verified`, `behavior_verified`, `inferred`
Match classes (Stage 3)	`exact`, `partial`, `incorrect`, `unmatched_prediction`, `unmatched_ground_truth`, `ambiguous`

Appendix D: Kiro quick reference for this exercise

Task	What to do in Kiro
Start a stage	Chat panel > new session (Vibe) > paste the stage prompt
Limit scope	Type `#` and attach `#Folder` / `#File` for the allowed inputs only
Review changes as they happen	Use Supervised mode
Let the agent run unattended	Use Autopilot mode (after you trust the stage)
Reduce approval prompts	Trust safe commands such as `dotnet build`, `dotnet test`, `git status`
Undo a bad run	Revert the agent's changes in Kiro, or `git checkout -- .` / `git reset --hard HEAD` on your learner branch
Resume a long build	New session > "Continue ... read implementation-progress.md ..."

Third-party names are used for identification only. The legacy portfolio uses fictitious people and organizations. Kiro features and UI labels may change; see https://kiro.dev for current documentation.